



WIRELESS VEHICLE (V2V) COMMUNICATIONS

THE UNIVERSITY OF TEXAS AT DALLAS

MEMBERS: RICARDO ALANIS, LUCAS CASTRO, BRANDON MARZIK, BRADLEY
OLBREY

SPONSOR: ROLAND VARRIALE

OVERVIEW



BACKGROUND



PROJECT
FOCUS



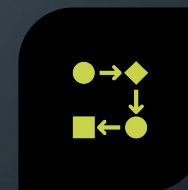
APPROACH



ISSUES
ENCOUNTERED



RESULTS



CONCLUSIONS

VEHICLE TO VEHICLE COMMUNICATION



With the increase of autonomous cars and smarter vehicles the protocol they use to communicate critical information needs to be defined.



Vehicles would be able to exchange speed information, location information, direction, and emergency information while in operation.



These operations would later be pulled into exchanging information with the infrastructure of the road and information pulled from pedestrians.

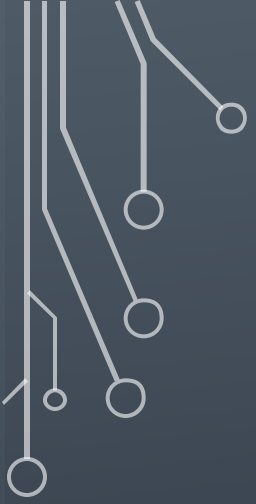
DEDICATED SHORT-RANGE COMMUNICATION

DSRC is one-way or two-way short-range to medium-range wireless communication channels specifically designed for automotive use.

An amendment was made to the IEEE 802.11 standard to include enhancements to support intelligent transportation systems (ITS).

This involves data exchanges between high speed vehicles and the road side infrastructure using DSRC.

This amendment licenses the 5.9 GHz band (5.85-5.925 GHz) to be dedicated for this purpose. This is also known as 802.11p.



PROJECT FOCUS



- The 802.11p specification in the IEEE standard is pretty bare at the moment.
- There are no security specifications for the protocol currently built into it.
- Our goal is to attempt an a basic DOS attack to see the practicality of an attack against the V2X system.

THREAT SCOPE

The attacker has no ability to modify any system currently operating in the ecosystem.

- Has no physical access or wireless access to any system i.e. cars, RSU, traffic lights etc.

The attacker can only introduce new devices to the network that add information to the network.

- Can introduce their own RSU to the system.

APPROACH



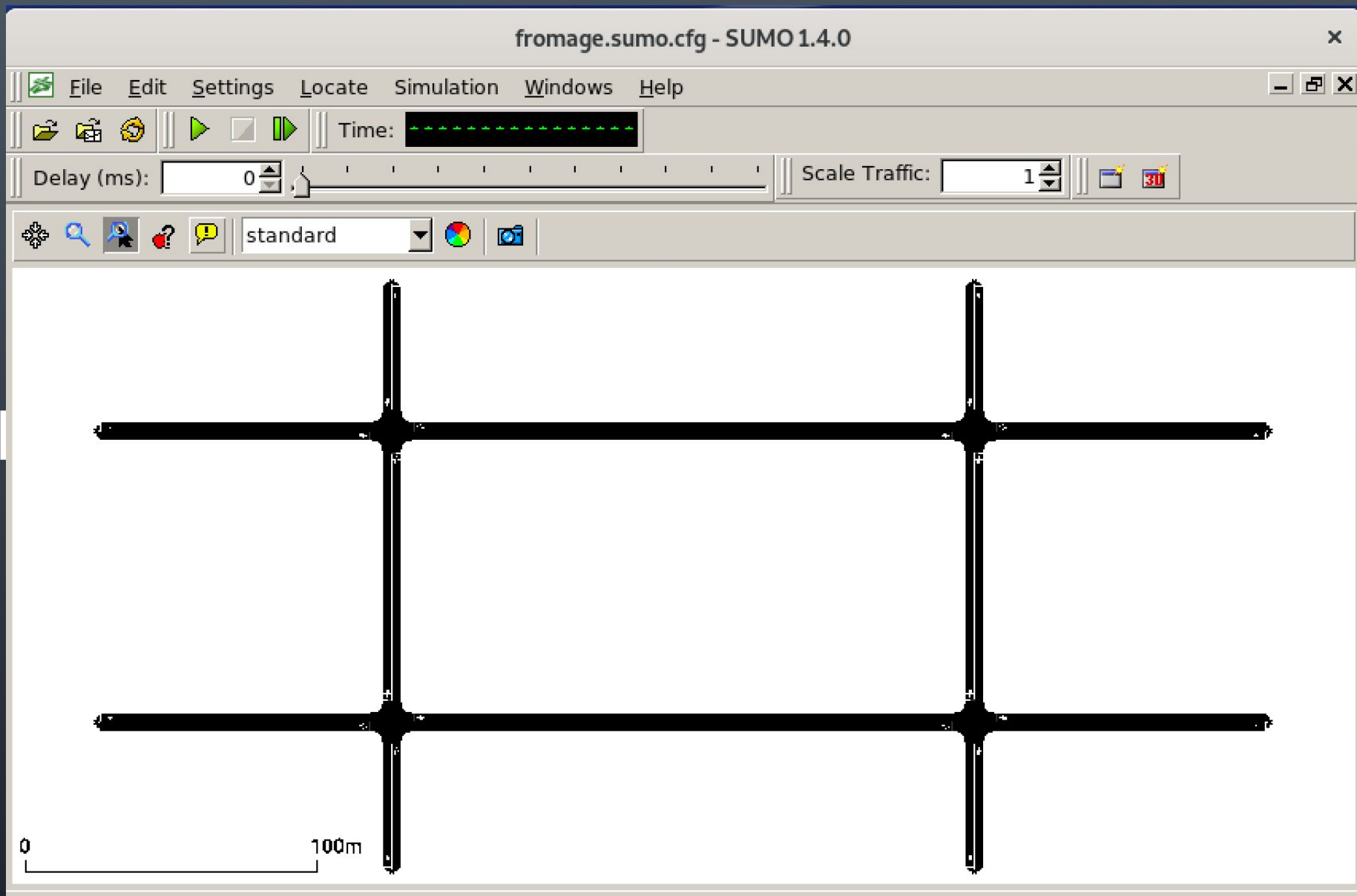
Use the Veins framework with OMNeT++ to simulate an attack against wireless communicating vehicles.



Introduce a new device to the network (malicious road side unit) to conduct an attack inside the ecosystem.



Send a large amount of messages to overload the network using the RSU to perform a DoS attack.



Activitiesapp

Tue 02:04

OMNeT++/Qtenv (release) - WithBeaconing #0 - omnetpp.ini - /home/veins/maliciousveins/maliciousveins/examples/omnetdufromage

FileSimulateInspectViewHelp

STEP

RUN

FAST

EXPRESS

UNTIL...

STOP

PAUSE

REC

Q

☰

☐

→

⚙

●

▬

next: #4178s 844ms 277us 476ns 545ps

Next: (veins::AirFrame11p, id=721)

In: Scenario.node[0].nic.phy80211p (PhyLayer80211p, id=17)

At: 8.84427762029s (now+0.000000143745s)

[]

+0s

+100ps

+1ns

+10ns

+100ns

+1us

+10us

+100us

+1ms

+10ms

+100ms

+1s

+10s

+100s

Scenario (Scenario) id=1

scheduled-events (cEventHeap)

world

connectionManager

node[0]

(AirFrame11p)

obstacles

annotations

node[1]

(AirFrame11p)

rsu[0]

(AirFrame11p)

node[2]

manager

Zoom:2.20x

Scenario (Scenario) id=1

playgroundSizeX (cPar) 2500

playgroundSizeY (cPar) 2500

playgroundSizeZ (cPar) 50m

obstacles (ObstacleControl) id=5

annotations (AnnotationManager)

connectionManager (ConnectionManager)

world (BaseWorldUtility) id=5

manager (TraCIScenarioManager)

rsu[0] (RSU) id=7

canvas (cCanvas) 2 toplevel f

node[0] (Car) id=13

node[1] (Car) id=19

node[2] (Car) id=25

** Event #386 t=8.623563696397 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on (veins::DemoSafetyMessage, id=668)

** Event #387 t=8.623568002976 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on selfmsg next Mac Event (omnetpp::cMessage, id=59)

** Event #388 t=8.623568002976 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on Radio switching over (omnetpp::cMessage, id=672)

** Event #389 t=8.623569002976 Scenario.node[0].nic.phy80211p (PhyLayer80211p, id=17) on (veins::Mac80211Pkt, id=670)

** Event #390 t=8.623569311633 Scenario.rsu[0].nic.phy80211p (PhyLayer80211p, id=11) on (veins::AirFrame11p, id=677)

** Event #391 t=8.623569311633 Scenario.rsu[0].nic.mac1609_4 (Mac1609_4, id=12) on ChannelStatus (omnetpp::cMessage, id=682)

** Event #392 t=8.623569671112 Scenario.node[1].nic.phy80211p (PhyLayer80211p, id=23) on (veins::AirFrame11p, id=679)

** Event #393 t=8.623569671112 Scenario.node[1].nic.mac1609_4 (Mac1609_4, id=24) on ChannelStatus (omnetpp::cMessage, id=684)

** Event #394 t=8.623570002235 Scenario.node[2].nic.phy80211p (PhyLayer80211p, id=29) on (veins::AirFrame11p, id=676)

** Event #395 t=8.623570002235 Scenario.node[2].nic.mac1609_4 (Mac1609_4, id=30) on ChannelStatus (omnetpp::cMessage, id=686)

** Event #396 t=8.623673002976 Scenario.node[0].nic.phy80211p (PhyLayer80211p, id=17) on selfmsg transmission over (omnetpp::cMessage, id=49)

** Event #397 t=8.623673002976 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on Transmission over (omnetpp::cMessage, id=688)

** Event #398 t=8.623673002976 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on ChannelStatus (omnetpp::cMessage, id=690)

** Event #399 t=8.623673002976 Scenario.node[0].nic.mac1609_4 (Mac1609_4, id=18) on Radio switching over (omnetpp::cMessage, id=692)

** Event #400 t=8.623673311633 Scenario.rsu[0].nic.phy80211p (PhyLayer80211p, id=11) on selfmsg (veins::AirFrame11p, id=677)

** Event #401 t=8.623673311633 Scenario.rsu[0].nic.mac1609_4 (Mac1609_4, id=12) on (veins::Mac80211Pkt, id=694)

** Event #402 t=8.623673311633 Scenario.rsu[0].nic.mac1609_4 (Mac1609_4, id=12) on ChannelStatus (omnetpp::cMessage, id=696)

** Event #403 t=8.623673311633 Scenario.rsu[0].appl (maliciousRSU, id=8) on (veins::DemoSafetyMessage, id=698)

** Event #404 t=8.623673671112 Scenario.node[1].nic.phy80211p (PhyLayer80211p, id=23) on selfmsg (veins::AirFrame11p, id=679)

** Event #405 t=8.623673671112 Scenario.node[1].nic.mac1609_4 (Mac1609_4, id=24) on (veins::Mac80211Pkt, id=700)

** Event #406 t=8.623673671112 Scenario.node[1].nic.mac1609_4 (Mac1609_4, id=24) on ChannelStatus (omnetpp::cMessage, id=702)

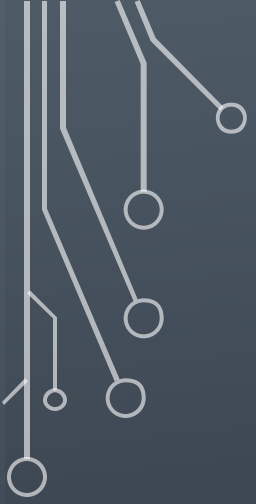
** Event #407 t=8.623673671112 Scenario.node[1].appl (ApplicationLayerTest, id=20) on (veins::DemoSafetyMessage, id=704)

** Event #408 t=8.623674002235 Scenario.node[2].nic.phy80211p (PhyLayer80211p, id=29) on selfmsg (veins::AirFrame11p, id=676)

** Event #409 t=8.623674002235 Scenario.node[2].nic.mac1609_4 (Mac1609_4, id=30) on (veins::Mac80211Pkt, id=706)

** Event #410 t=8.623674002235 Scenario.node[2].nic.mac1609_4 (Mac1609_4, id=30) on ChannelStatus (omnetpp::cMessage, id=708)

** Event #411 t=8.623674002235 Scenario.node[2].appl (ApplicationLayerTest, id=26) on (veins::DemoSafetyMessage, id=710)



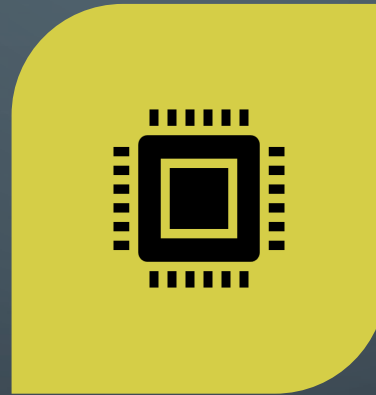
ISSUES ENCOUNTERED

- Not a lot of documentation or examples for using Veins.
 - OMNeT++ is not very straight forward.
 - Performing a DoS attack on a single-threaded application presents problems.
- 

RESULTS



WORKING V2X SIMULATION
STOOD UP



ATTEMPTED DOS ATTACK
WITHIN SIMULATION



SINGLE-THREADED NATURE
AND DETERMINISTIC NATURE
OF OMNET++ PREVENTED
SIMULATION OF DOS ATTACK

CONCLUSIONS

- Learned about OMNeT++ and VeINS framework.
- Learned how to set up a SUMO traffic simulation.
- Got a working simulation working under normal conditions.
- Know how to setup a DOS attack against wireless vehicles in a non deterministic setting.

CONCLUSIONS

- OMNeT++/Veins not necessarily ideal for security testing
- Single-threaded by default, any multithreading would require rewriting of both OMNeT++ and Veins
- Deterministic systems don't allow unexpected things to go wrong. Most attacks makes systems do unexpected things
- DoS attack still theoretically possible

REFERENCES

- [https://en.wikipedia.org/wiki/Dedicated short-range communications](https://en.wikipedia.org/wiki/Dedicated_short-range_communications)
- <https://www.extremetech.com/extreme/176093-v2v-what-are-vehicle-to-vehicle-communications-and-how-does-it-work>